

1. Who is a Backend Developer?

A Backend Developer designs, builds, and maintains the server-side logic, APIs, databases, and infrastructure behind applications.

2. Responsibilities

Build APIs, design databases, implement authentication, optimize performance, write tests, deploy services, monitor production, and collaborate with frontend, DevOps, and product teams.

3. Required Skills

Mandatory: Programming language (Python/Java/Node.js/Go/C#), SQL, REST APIs, Git, backend framework.

Preferred: Docker, Redis, message queues, CI/CD, cloud, monitoring, system design.

4. Learning Roadmap

Phase 1: Language, Git, SQL, HTTP (Goal: fundamentals; Time: 1–2 months; Outcome: CRUD apps).

Phase 2: Framework + REST APIs (Outcome: authenticated APIs).

Phase 3: Databases, caching, testing, deployment.

Phase 4: System design and cloud.

5. Projects

Beginner: Task API (Skills: CRUD, SQL).

Intermediate: Auth service, E-commerce backend (Skills: JWT, PostgreSQL, testing).

Advanced: Microservices platform (Skills: Docker, Redis, CI/CD, cloud).

6. Resume Optimization Tips

Show measurable impact, deployed APIs, database design, testing, cloud deployments, and GitHub links.

7. Common ATS Mistakes

Missing backend keywords, poor formatting, generic summaries, no metrics, no deployment experience.

8. Interview Topics

Junior: HTTP, REST, SQL, Git.

Intermediate: Authentication, caching, Docker, testing.

Senior: System design, scalability, distributed systems, observability.

9. Common Industry Tech Stack

FastAPI/Spring Boot/Express/ASP.NET Core • PostgreSQL • Redis • Docker • Nginx • GitHub Actions • AWS • Prometheus/Grafana.

10. Career Progression

Junior → Backend Developer → Senior → Lead/Architect, with increasing expectations in frameworks, cloud, system design, scalability, and leadership.

11. Backend Architecture Concepts

Client–Server Architecture: Separation of frontend and backend responsibilities.

Monolith vs Microservices: Single deployable application versus independently deployable services.

REST & GraphQL: Common API architectural styles for communication.

Authentication & Authorization: JWT, OAuth, sessions, role-based access control.

Caching: Redis, in-memory caching, CDN strategies to reduce latency.

Load Balancing: Distribute requests across multiple servers for scalability and availability.

Message Queues: RabbitMQ, Kafka, or similar systems for asynchronous processing.

Database Scaling: Indexing, replication, sharding, connection pooling.

Observability: Logging, metrics, tracing, monitoring, and alerting.

Resilience: Retry logic, circuit breakers, rate limiting, graceful failure handling.